

Project Description

AI Workspace Agent Suite

Google Workspace MCP + workspace-cli + LangGraph + OpenAI GPT-4o

1. Project Overview

This project is a suite of two autonomous AI agents that connect to a live Google Workspace account and handle real-world business tasks — reading and replying to customer refund emails, and managing Google Calendar — entirely through natural language.

Both agents share the same core architecture: the **ReAct (Reasoning + Acting)** pattern, implemented with **LangGraph StateGraph**, powered by **OpenAI GPT-4o**, and connected to Google Workspace through the open-source `google_workspace_mcp` server (github.com/taylorwilsdon/google_workspace_mcp).

The Calendar Agent adds a second tool layer on top of MCP: `workspace-cli` **bash tools** — direct subprocess calls to the `workspace-cli` command-line interface that ship with the same repo. This gives the agent two ways to read calendar data: fast, lightweight CLI calls for simple queries, and full MCP tool calls for creating, updating, or deleting events.

AI Workspace Agent Suite	
Refund Email Agent (refund_agent.py)	Calendar Agent (calendar_agent.py)
LangGraph ReAct Graph agent_node ↔ tool_node (ReAct loop)	
OpenAI GPT-4o (reasoning LLM)	
Google Workspace MCP Server (workspace-mcp, local stdio) workspace-cli (bash subprocess, Calendar Agent only)	
Google APIs (Gmail API + Google Calendar API) OAuth 2.0 (Google Cloud credentials)	

2. Projects in the Suite

Project A — Refund Email Agent (`refund_agent.py`)

An autonomous customer service agent that monitors a Gmail inbox for refund and return request emails, classifies them, composes professional replies using pre-defined templates, and sends threaded replies back to customers — without any human involvement beyond the initial run command.

Automated 6-step workflow:

```
SEARCH inbox → READ each email → CLASSIFY intent →  
DRAFT reply from template → SEND threaded reply → REPORT summary
```

Email classifications handled:

Class	Description	Agent Action
REFUND_REQUEST	Customer wants money back	Send refund approval reply (3–5 day processing)
RETURN_REQUEST	Customer wants to return product	Send return instructions with prepaid label steps
COMPLAINT	General dissatisfaction	Send empathetic acknowledgement, 24hr follow-up promise
OTHER	Unrelated content	Skip — no reply sent

Project B — Calendar Agent (`calendar_agent.py`)

An interactive AI assistant that answers natural language questions about a Google Calendar, creates and modifies events, checks for free time slots, and sends RSVPs — using both MCP tools and lightweight CLI bash tools depending on the complexity of the query.

Dual tool strategy:

```
Simple read → workspace-cli bash tool (fast subprocess, minimal overhead)
```

3. Technology Stack

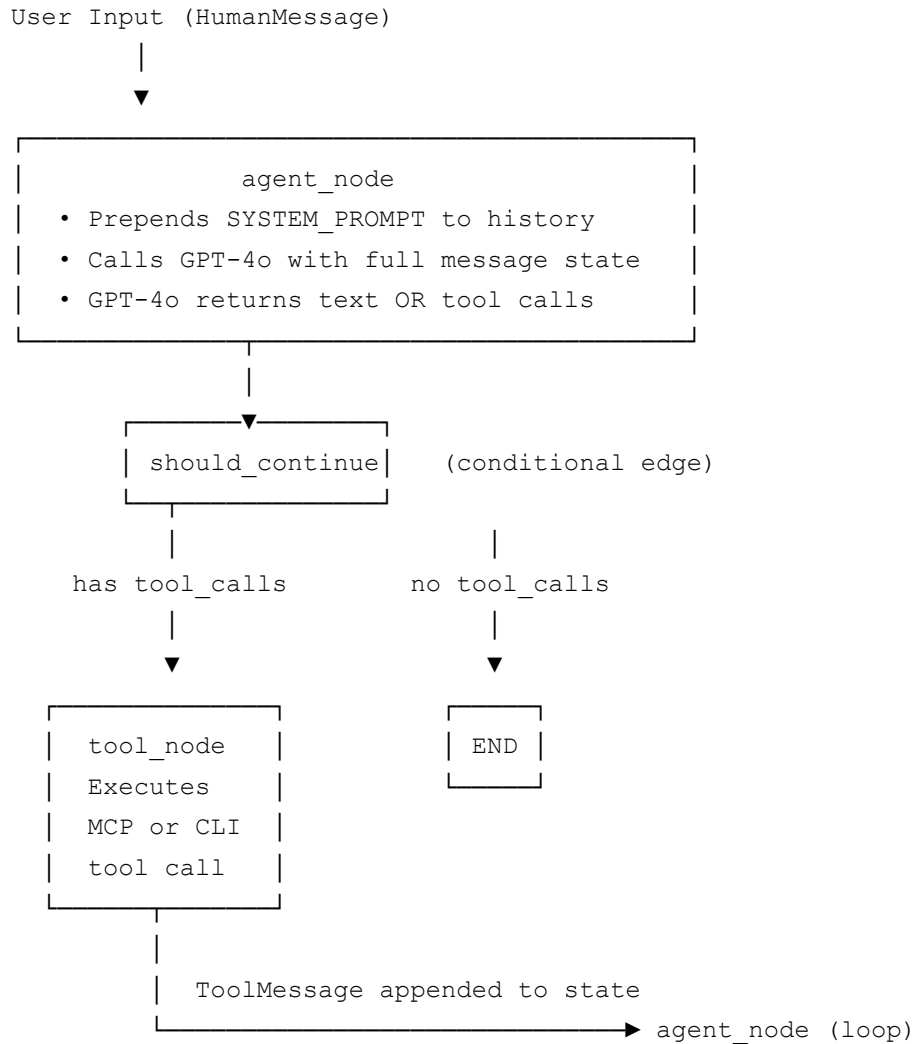
Component	Technology	Version / Source
Language	Python	3.11+
LLM	OpenAI GPT-4o	gpt-4o via langchain-openai
Agent Framework	LangGraph	StateGraph + ToolNode
Tool Protocol	Model Context Protocol (MCP)	Open standard, Linux Foundation
MCP Server	google_workspace_mcp	github.com/taylorwilsdon/google_workspace_mcp
CLI Tool	workspace-cli	Built into same repo, installed via uv tool install .
Gmail Access	Google Gmail API	OAuth 2.0 scoped permissions
Calendar Access	Google Calendar API	OAuth 2.0 scoped permissions
Auth	Google Cloud OAuth 2.0	Desktop App flow
Transport	stdio (local subprocess)	MCP JSON-RPC over stdin/stdout

Python packages:

```
langgraph
langchain-openai
langchain-mcp-adapters
langchain-core
workspace-mcp
```

4. System Architecture — ReAct Graph

Both agents share the same three-node LangGraph graph topology:



Key principle: The loop continues until GPT-4o produces a response with no tool calls. A single user question may trigger 5–15 tool calls internally before the agent produces its final answer.

5. Tool Inventory

5.1 Gmail MCP Tools — Refund Email Agent

These tools are loaded from the running `workspace-mcp` server via `mcp_client.get_tools()` and filtered by name.

Tool Name	Purpose	Key Parameters
	Search inbox with Gmail	<code>query</code> ,

search_gmail_messages	query operators	max_results
get_gmail_message_content	Read full body + metadata of one email	message_id
get_gmail_messages_content_batch	Fetch up to 25 emails in one call	message_ids , format
send_gmail_message	Send or reply to an email (threaded)	to , subject , body , thread_id
create_gmail_draft	Save a draft for human review	to , subject , body
get_gmail_thread	Read full conversation thread	thread_id
list_gmail_labels	List all Gmail labels and folders	(none)

5.2 Calendar MCP Tools — Calendar Agent

Tool Name	Purpose	Key Parameters
list_calendar_events	List events in a date range	calendarId , timeMin , timeMax
get_calendar_event	Get one event by ID	calendarId , eventId
list_calendars	List all calendars the user has	(none)
create_calendar_event	Create a new calendar event	summary , start , end , attendees
update_calendar_event	Update an existing event	calendarId , eventId , updates
delete_calendar_event	Delete an event	calendarId , eventId
suggest_meeting_time	Find free slots across attendees	attendees , duration
respond_to_calendar_event	RSVP accept / decline /	calendarId , eventId ,

	tentative	response
--	-----------	----------

5.3 workspace-cli Bash Tools — Calendar Agent Only

These are Python `@tool`-decorated functions that call `workspace-cli` as a local subprocess. They parse the structured JSON output from the CLI and return it to the agent. Used for fast, low-overhead calendar reads.

Tool Name	CLI Command Executed	Use Case
<code>cli_today_events</code>	<code>workspace-cli call list_calendar_events (today's ISO range)</code>	"What's on today?"
<code>cli_list_events</code>	<code>workspace-cli call list_calendar_events (custom range)</code>	"Show me this week"
<code>cli_list_calendars</code>	<code>workspace-cli call list_calendars</code>	"What calendars do I have?"
<code>cli_get_event</code>	<code>workspace-cli call get_calendar_event --eventId <id></code>	"Get details for that meeting"
<code>cli_tool_list</code>	<code>workspace-cli list</code>	Debug / tool discovery

Why both MCP and CLI?

CLI tools: no MCP roundtrip, instant subprocess, ideal for read-only queries
MCP tools: full CRUD, richer error handling, ideal for create/update/delete

GPT-4o selects the appropriate tool surface based on the task type.

6. Component Descriptions

6.1 AgentState — Shared Memory (Both Agents)

```
class AgentState(TypedDict):  
    messages: Annotated[Sequence[BaseMessage], add_messages]
```

A Python `TypedDict` that holds the full conversation history for the running agent session. The `add_messages` reducer appends new messages rather than replacing the list, so every

tool result, AI response, and user message accumulates across the entire ReAct loop. This is the “working memory” that allows multi-step reasoning to function — without it, the agent forgets what it did on every step.

6.2 `WORKSPACE_MCP_CONFIG` — MCP Server Config (Both Agents)

```
WORKSPACE_MCP_CONFIG = {
    "workspace": {
        "command": "uvx",
        "args": ["workspace-mcp", "--single-user", "--tool-tier", "core",
                "--permissions", "gmail:send"], # or "calendar"
        "transport": "stdio",
        "env": { "GOOGLE_OAUTH_CLIENT_ID": ..., "GOOGLE_OAUTH_CLIENT_SECRET": ...
    }
}
```

Tells `MultiServerMCPClient` how to spawn and communicate with the `workspace-mcp` subprocess. Key design decisions:

- `--single-user` — OAuth 2.0 for one Google account
 - `--tool-tier core` — loads only essential tools, not the full 100+
 - `--permissions gmail:send or calendar` — restricts scope to only what the agent needs
 - `transport: stdio` — local pipe, no network, data never leaves the machine
 - OAuth credentials injected via `env` from shell environment variables
-

6.3 `SYSTEM_PROMPT` — Persona + Workflow Instructions (Both Agents)

A `SystemMessage` prepended to every LLM call that defines the agent’s identity, available tools, decision rules, reply templates (email agent), tool selection guide (calendar agent), and output formatting requirements.

Refund agent prompt — defines the 6-step workflow, three reply templates (REFUND / RETURN / COMPLAINT), and hard rules including: always thread replies using `thread_id`, never reply to OTHER emails, prefer `create_gmail_draft` over direct sends when uncertain.

Calendar agent prompt — includes today’s date, explains the CLI vs MCP tool selection guide, output formatting rules (ISO → human-readable time), and confirmation requirements

before destructive operations (delete / update).

The system prompt is the difference between a generic GPT-4o and a focused, reliable domain agent. It encodes institutional knowledge that would otherwise require human judgment.

6.4 `_run_cli(args)` — CLI Subprocess Runner (Calendar Agent)

```
def _run_cli(args: list[str], timeout: int = 15) -> dict[str, Any]
```

A private synchronous helper that runs `workspace-cli <args>` as a subprocess using Python's `subprocess.run()`. It captures stdout, attempts `json.loads()` on the output (since `workspace-cli` returns structured JSON), and falls back to raw text if parsing fails. Handles three error conditions: non-zero exit code, timeout (default 15s), and

`FileNotFoundError` when `workspace-cli` is not installed.

All five CLI `@tool` functions delegate to `_run_cli` — they only differ in the arguments they pass and the docstring they expose to GPT-4o for tool selection.

6.5 CLI `@tool` Functions — Calendar Agent

Five Python functions decorated with `@tool` from `langchain_core.tools`. The decorator exposes the function's name, type signature, and docstring to GPT-4o as a callable tool schema.

- `cli_today_events(calendar_id)` — computes today's UTC start/end timestamps and calls `_run_cli` with `list_calendar_events` scoped to that range. No date arguments needed from the user.
- `cli_list_events(time_min, time_max, max_results, calendar_id)` — general-purpose event lister with default `time_max` of 7 days from now if not provided. Passes `singleEvents=true` and `orderBy=startTime` for clean chronological output.
- `cli_list_calendars()` — calls `list_calendars` with no arguments, returns all calendar metadata including ID, name, access role, and color.
- `cli_get_event(event_id, calendar_id)` — fetches a single event's full details. Event IDs come from the output of `cli_list_events` or `cli_today_events`.
- `cli_tool_list()` — calls `workspace-cli list` to enumerate all tools the running server exposes. Useful for debugging version mismatches or discovering new tool

names after a server update.

6.6 `build_agent(mcp_client)` — Agent Factory (Both Agents)

```
async def build_agent(mcp_client: MultiServerMCPClient) -> CompiledGraph
```

Assembles the complete LangGraph agent in five steps:

1. Calls `mcp_client.get_tools()` to fetch the live tool list from the running MCP server subprocess
2. Filters tools by name to the relevant set (Gmail tools or Calendar tools)
3. Merges MCP tools with CLI `@tool` functions (Calendar Agent only)
4. Creates `ChatOpenAI(model="gpt-4o")` and binds all tools via `.bind_tools(all_tools)` — enabling GPT-4o's native function calling
5. Defines nodes (`agent_node`, `tool_node`), edges, and conditional routing, then calls `.compile()` to produce the runnable graph

Returns a compiled LangGraph graph ready to accept `{"messages": [...]}` input.

6.7 `agent_node(state)` — LLM Reasoning Node (Both Agents)

```
def agent_node(state: AgentState) -> AgentState
```

Defined inside `build_agent` with closure access to the bound LLM. On every invocation it constructs `[SYSTEM_PROMPT] + list(state["messages"])` and calls `llm.invoke()`. GPT-4o reads the full conversation history — including all previous tool results — and produces either a tool call (continue looping) or a plain text response (end the graph). The response is returned as `{"messages": [response]}` and appended to state by the `add_messages` reducer.

6.8 `should_continue(state)` — Routing Function (Both Agents)

```
def should_continue(state: AgentState) -> str # "tools" | END
```

The conditional edge function. Inspects the last message in state: if it is an `AIMessage` with non-empty `tool_calls`, returns `"tools"` to route to `tool_node`. Otherwise returns `END` to terminate. This is the function that implements the ReAct loop — it decides whether the agent has finished thinking or needs to act again.

6.9 `tool_node` — MCP + CLI Executor (Both Agents)

```
tool_node = ToolNode(all_tools)
```

A prebuilt LangGraph node that receives structured tool call JSON from `agent_node`, dispatches to the correct tool (MCP tool via the MCP client, or CLI `@tool` function via direct Python call), wraps the result in a `ToolMessage`, and appends it to state. LangGraph handles JSON serialisation, error wrapping, and the MCP client's async communication protocol automatically.

For the Calendar Agent, `ToolNode` handles both tool surfaces transparently — GPT-4o does not know or care whether a tool is backed by MCP or a subprocess.

6.10 `run_auto_refund_processing(agent)` — Auto Mode (Refund Agent)

```
async def run_auto_refund_processing(agent) -> None
```

Fires one fixed `HumanMessage` instructing the agent to execute the complete 6-step workflow end-to-end. Calls `agent.ainvoke()` and waits for the full ReAct loop to finish (typically 10–20 tool calls). Extracts the final `AIMessage` from the result and prints the summary report. No further user input is required — fully autonomous.

6.11 `run_demo(agent)` — Demo Mode (Calendar Agent)

```
async def run_demo(agent) -> None
```

Iterates over three pre-written demo queries: "What calendars do I have?", "What's on my calendar today?", "Show me my events for the next 7 days." — invoking the agent independently for each, printing numbered output. Demonstrates the agent's CLI and MCP tool selection without requiring user input. Useful for testing that OAuth credentials and tool connections are working.

6.12 `run_interactive_chat(agent)` — Interactive Mode (Both Agents)

```
async def run_interactive_chat(agent) -> None
```

Runs a `while True` CLI input loop. Maintains a `history` list of `BaseMessage` objects that grows with each turn, enabling multi-turn conversation where the agent remembers earlier context. Passes the full `history` into every `agent.ainvoke()` call. Exits on `"exit"` or `"quit"`. Calendar Agent additionally handles `"demo"` as a special command that runs `run_demo()` inline.

6.13 `main()` — Orchestrator (Both Agents)

```
async def main() -> None
```

Top-level entry point. Validates required environment variables, exits with the setup guide if any are missing, opens `MultiServerMCPClient` as an `async` with context manager (which spawns and keeps alive the `workspace-mcp` subprocess for the session), calls `build_agent()`, then routes to the appropriate run mode based on user input.

6.14 `_print_setup_guide()` — Developer Helper (Both Agents)

```
def _print_setup_guide() -> None
```

Prints a complete terminal guide covering installation, Google Cloud OAuth setup, scope configuration, environment variable export, and example CLI verification commands. Called only when env var validation fails.

7. Data Flow — End-to-End Example

Refund Agent: "Process all refund emails"

1. `main()`
 - └─ `MultiServerMCPClient` starts `workspace-mcp` subprocess
2. `agent_node (turn 1)`

- └ GPT-4o: "I need to search Gmail for refund emails"
 - └ Emits: tool_call → search_gmail_messages(query="refund OR return is:unread")
3. tool_node
- └ MCP client → workspace-mcp → Gmail API
 - └ Returns: [{id: "abc123", subject: "Refund for Order #4892"}, ...]
4. agent_node (turn 2)
- └ GPT-4o: "Found 3 emails. Reading first one."
 - └ Emits: tool_call → get_gmail_message_content(message_id="abc123")
5. tool_node
- └ Returns: full email body, sender, thread_id
6. agent_node (turn 3)
- └ GPT-4o: "This is a REFUND_REQUEST. Composing reply."
 - └ Emits: tool_call → send_gmail_message(to=..., thread_id=..., body=...)
7. tool_node
- └ Email sent. Returns: {id: "sent_xyz", labelIds: ["SENT"]}
8. agent_node (turns 4-N)
- └ Repeats steps 4-7 for remaining 2 emails
9. agent_node (final turn)
- └ GPT-4o: no more tool_calls
 - └ Returns: "Processed 3 emails: 2 REFUND, 1 RETURN. All replied."
10. should_continue → END

Calendar Agent: "What's on today?"

1. agent_node (turn 1)
- └ GPT-4o: "Simple read – use CLI tool for speed"
 - └ Emits: tool_call → cli_today_events(calendar_id="primary")
2. tool_node
- └ _run_cli(["call", "list_calendar_events", "--timeMin", ...])
 - └ subprocess: workspace-cli → Google Calendar API
 - └ Returns: JSON with today's events
3. agent_node (turn 2)
- └ GPT-4o: no more tool_calls
 - └ Returns: "You have 3 events today: 9:00 AM Standup, ..."
4. should_continue → END

8. Security Design

Concern	How It Is Addressed
OAuth credentials	Stored in env vars, never hardcoded. Tokens cached encrypted at <code>~/.workspace-mcp/</code> using a Fernet key.
Permission scope	<code>--permissions gmail:send or calendar</code> — each agent loads only what it needs. No admin, no Drive writes.
Data locality	MCP server runs locally via stdio. Gmail/Calendar data never passes through a third-party server.
Auto-send guard	Refund agent system prompt instructs: prefer <code>create_gmail_draft</code> over <code>send_gmail_message</code> when uncertain.
CLI timeout	<code>_run_cli()</code> enforces a 15-second timeout on every subprocess call to prevent hanging.
Destructive ops	Calendar agent system prompt requires explicit confirmation before delete or update operations.

9. Setup Instructions

```
# Step 1 - Clone and install workspace-mcp + CLI
git clone https://github.com/taylorwilsdon/google_workspace_mcp
cd google_workspace_mcp
uv tool install . # installs workspace-cli globally
pip install workspace-mcp

# Step 2 - Install Python dependencies
pip install langgraph langchain-openai langchain-mcp-adapters

# Step 3 - Google Cloud OAuth setup
# a) console.cloud.google.com → New project
# b) Enable: Gmail API + Google Calendar API
# c) OAuth consent screen → External → add your email as test user
# d) Scopes: gmail.modify, gmail.send, calendar, calendar.events
# e) Create OAuth 2.0 credentials → Desktop app
```

```
# f) Copy Client ID and Client Secret

# Step 4 - Export environment variables
export OPENAI_API_KEY=sk-...
export GOOGLE_OAUTH_CLIENT_ID=<your-client-id>
export GOOGLE_OAUTH_CLIENT_SECRET=<your-client-secret>
export OAUTHLIB_INSECURE_TRANSPORT=1      # local dev only

# Step 5 - Verify CLI works before running agents
workspace-cli list                        # list all available tools
workspace-cli call list_calendars        # verify Google OAuth works

# Step 6 - Run either agent
python refund_agent.py      # customer service email processing
python calendar_agent.py    # calendar query and management
```

10. Example Interactions

Refund Agent (auto mode):



Auto-processing refund & return emails...

Agent Summary:

Found 4 unread emails matching refund/return query.

- alice@email.com - "Where is my refund?" → REFUND_REQUEST → Replied ✓
- bob@email.com - "I want to return my item" → RETURN_REQUEST → Replied ✓
- carol@email.com - "This is unacceptable" → COMPLAINT → Replied ✓
- info@promo.com - "Special offer inside" → OTHER → Skipped

Calendar Agent (interactive mode):

You: What's on my calendar today?

Agent: You have 3 events today (Saturday, May 16):

- 9:00 AM - Team Standup (Google Meet)
- 2:00 PM - Client Review (Conference Room B)
- 5:30 PM - 1:1 with Manager

You: Schedule a team lunch for next Friday at noon for 1 hour

Agent: Created: "Team Lunch" on Friday May 23, 12:00 PM - 1:00 PM ✓

You: Find a free 30-minute slot for a call with john@example.com this week

Agent: Checking availability... Available slots:

- Tuesday May 20 10:00-10:30 AM

- Wednesday May 21 3:00-3:30 PM
 - Thursday May 22 11:00-11:30 AM
- Which would you like to book?

11. Key Concepts Reference

Concept	Definition	Where Used
ReAct pattern	Reason → Act → Observe loop; agent alternates thinking and tool use	Both agents
LangGraph StateGraph	Directed graph of nodes (functions) connected by typed edges	<code>build_agent()</code>
MCP (Model Context Protocol)	Open standard JSON-RPC protocol for AI tool access	<code>WORKSPACE_MCP_CONFIG</code>
Tool binding	Attaching tool schemas to an LLM so it can emit structured function calls	<code>llm.bind_tools()</code>
stdio transport	MCP communication via stdin/stdout pipe — runs entirely locally	MCP config
TypedDict	Python type hint for dicts with fixed keys; used for state schema	<code>AgentState</code>
<code>add_messages</code> reducer	LangGraph helper that appends to the message list instead of replacing it	<code>AgentState</code>
Conditional edge	LangGraph routing that dynamically chooses the next node based on state	<code>should_continue()</code>
OAuth 2.0	Delegated access standard; user grants scoped permissions without sharing password	Google Cloud setup
Fernet encryption	Symmetric encryption used by <code>workspace-cli</code> for local token caching	Token storage
Thread ID	Gmail identifier that groups messages in the same email conversation	<code>send_gmail_message</code>

tool_tier core	workspace-mcp flag that loads ~20 essential tools instead of all 100+	MCP config
@tool decorator	LangChain decorator that converts a Python function into an LLM-callable tool	CLI bash tools
subprocess	Python module used to spawn and communicate with the <code>workspace-cli</code> process	<code>_run_cli()</code>

MCP Server: github.com/taylorwilsdon/google_workspace_mcp — MIT License **LangGraph:** langchain-ai.github.io/langgraph **MCP Spec:** modelcontextprotocol.io **OpenAI:** platform.openai.com/docs